

Automatic Generation and Integration of Tensor Graph Rewrites

Final Project Report of 2025 Fall UIUC CS 521

Heng Zhong
hengz2@illinois.edu
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA

Bhavya Hirani
bhirani2@illinois.edu
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA

Mengyao Wang
mengyao9@illinois.edu
University of Illinois
Urbana-Champaign
Champaign, Illinois, USA

Abstract

Many existing neural network frameworks and tensor compilers, e.g., XLA, optimize the computation of neural networks by manually designed tensor graph rewrites. TASO, however, provides a method to automatically synthesize and validate these rewrite rules.

Our project implements TASO’s synthesizer, a generation algorithm for tensor graph rewrites. Compared with the original Build algorithm in TASO, we defer the duplication checking to reduce the search space. The generated rules are validated by randomized testing instead of formal verification in the original paper. The cost of each graph pattern is measured by evaluating its execution time on random NumPy tensors.

After synthesis, the rewrite rules are automatically translated by our ONNX rule generator into rewrite rules in ONNX Script. This allows fully automated integration of these rules into existing neural network pipelines. Finally, we evaluate our project by its performance on real-world neural networks and provide an explanation for the results.

Keywords: tensor graph rewrites, tensor compilers, neural networks

1 Introduction

Modern deep learning frameworks adopt an intermediate format that represents neural networks (NN) as tensor computation graphs. In these graphs, each node represents an operator and each edge corresponds to a tensor value. To improve the execution speed of NNs, an important kind of optimization is tensor graph rewrite, which matches certain subgraph patterns in the original graph and replaces them with functionally equivalent subgraphs that have better performance.

Many existing deep learning frameworks, such as JAX/XLA [1, 8] and PyTorch [7], use manually designed graph rewrites. These rules improve the performance of NNS. However, the design and implementation of these rules require considerable human effort. For example, the algebraic simplifier of XLA contains more than 9k lines of C++ code, and many pattern-matching conditions are written ad hoc due to a

lack of structural representation. Furthermore, the implementations of rules are subject to human mistakes because there is no guarantee of the manual translation from their mathematical definition to program code.

TASO [3] addresses these two problems. It enumerates possible pattern graphs with a depth-first search algorithm. Candidate rewrite rules are first filtered by random testing, and then formally verified based on axiomatic properties of operators. To actually realize performance benefits, the TASO optimizer optimizes NNs with a cost model based on MetaFlow’s cost-based backtracking search [4].

Our project implements TASO’s Build algorithm for automatic synthesis of graph rewrite rules. The generated rules are then translated by our ONNX rule generator, which reduces the need for human efforts and the chance of errors. To simplify the task to a scale suitable for a one-month course project, we validate the correctness of rewrite rules only with random testing, and obtain the running time of graph patterns by testing their running time on NumPy.

The remaining contents of this report are as follows. Section 2 summarizes existing research and projects related to our work. Section 3 introduces the algorithms and system architecture in our design. Section 4 explains technical details of our project, including technical choices, major program components, and difficulties that we confronted during development. Finally, Section 5 presents our experiment and results.

2 Existing Work

Rewrite rule synthesis. The major part of our project is from TASO [3], which proposes fully-automated rewrite rule generation and verification methods. TASO first enumerates all possible graph patterns with a depth-first search that adds one operator node at a time, up to a threshold size. The graph patterns are then evaluated on some random but fixed integer tensors and categorized according to their hash. To further narrow down candidate rewrites, the patterns with same hash are tested on random floating-point tensors. After the generation, TASO formally verifies candidate rewrites with a set of predefined property axioms. These axioms describes

how different operators interoperate by giving the functional equivalence between some fundamental graph patterns.

To reduce the space of rewrite rules, TASO eliminates redundant rewrites. A rewrite rule is considered redundant if it is implied by some other rewrite rule by common sub-graphs or input renaming. Finally, TASO actually attains performance improvements by selecting graph rewrites with a cost-based backtracking search brought from MetaFlow [4].

Profitability and cost model. TASO uses MetaFlow [4] to compute the costs and decide the application sequence of graph rewrites. MetaFlow models costs as a multi-dimension value including FLOPS, kernel launches, and memory usage. After that, it applies rewrite rules with relaxed graph substitution that allows temporary cost degradation to attain an overall improvement.

Neural network framework. We choose ONNX [6] and ONNX Script [5] as the basis on which our project operates. ONNX is a uniform format for representing neural networks that enables synergy between different machine learning frameworks, such as PyTorch and TensorFlow. ONNX Script provides a more idiomatic way to create and transform ONNX models, including graph rewrites.

ONNX Script allows users to specify rewrite rules structurally, where each of the graph patterns and supplementary matching conditions is represented as an individual Python function. In contrast, many machine learning frameworks and compilers require ad hoc implementation of rewrite rules. For example, in XLA, the matching and substitution for a single rule are written as a single block of C++ code. We choose ONNX Script because its structural representation allows generating different components of a rule separately and as Python ASTs. This facilitates the modular design of our project’s architecture.

3 Design

Figure 1 illustrates the overall pipeline of our project. The rest of this section explains them in detail.

Graph pattern generation. Algorithm 1 shows how we generate possible graph patterns up to a threshold size. One improvement to note compared with the original Build algorithm of TASO is that we delay the check of duplicating (having same inputs) operators to the time they are added. Therefore, we only need to search matches for that operator instead of doing an n^2 traversal for every distinct G .

To support operators with parameters (e.g., `concat` and `split`), we regard each distinct parameter instantiation as a different operator, and enumerate their parameters up to a fixed upper bound, say, 4.

Split tree inference. We follow TASO’s treatment of `concat` and `split` operators, which assumes that all `split`’s are to

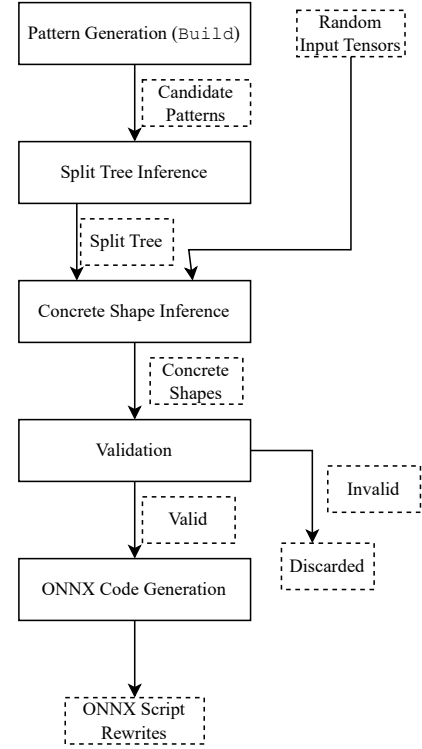


Figure 1. The pipeline of rewrite rule synthesis and ONNX code generation. *Pattern generation* enumerates all possible graph patterns with DFS up to a given size. *Split tree inference* deduces the axis and (symbolic) position of split operators. *Concrete shape inference* infers the numerical shapes of tensors for evaluation in validation. *Validation* refines candidate rewrites with hashing based on random but fixed integer inputs. Then it validates the functional equivalence of rewrites on random floating-point tensors. *ONNX code generation* generates executable ONNX Script rewrite rules as Python ASTs.

revert the effect of the most recent `concat`’s. To enable the extensibility to support more complicated operators that change ranks and dimensions, we designed the *split tree inference* pass. The pass numerically infers the split axis and symbolically infers the split index. This endows it dimension genericity. The complete rules are shown in Appendix B.

Rule validation. To restrict the workload within a one-month course project, we validate rewrite rules with random testing instead of formal verification. The validation is divided into 3 steps. First, we fix the shape of input tensors ($2 \times 4 \times 4$ in our implementation) and perform *concrete shape inference* on candidate graph patterns. This instantiates the symbolic split positions from split tree inference with actual integers. Then, the graph patterns are grouped according to the hash values of their evaluation results on some random but fixed integer tensors. Finally, rules within same groups

Algorithm 1 Graph pattern generation with DFS

```

1:  $n$ : Current graph size.
2:  $G$ : Currently built graph.
3:  $I$ : Available input tensors.
4:  $P$ : The set of all supported operators.
5:  $S$ : List of generated graphs.
6: function BUILD( $n : \mathbb{N}$ ,  $G : \text{Graph}$ ,  $I : \text{Tensor}$ )
7:    $D = D + G$ 
8:   if  $n < \text{threshold}$  then
9:     for  $op \in P$  do
10:      for  $i \in \phi(I)$  do //  $i$  a subset of inputs.
11:        if  $op(i)$  not duplicate in  $G$  then
12:          Add  $op(i)$  into graph  $G$ .
13:          Add outputs of  $G$  to  $I$ .
14:          BUILD( $n + 1$ ,  $G$ ,  $I$ )
15:          Remove  $op$  from  $G$ , outputs( $op$ )
16:          from  $I$ .
17: BUILD(1,  $\emptyset$ ,  $I$ )

```

are tested on random floating-point tensors. Pairs with equal outputs (up to an ϵ error) are kept as valid rewrite rules.

We pick profitable rules from the validated rules. However, the artifact of MetaFlow [2] is not readily integrable into our internal representation for graph generation. Therefore, considering the available time and workload, we use a one-dimensional cost model and greedy rule-application strategy.

ONNX Script code generation. To integrate the generated rewrites into current neural network workflow, we automatically translate them to Python ASTs representing ONNX rewrite rules. The target and replacement patterns are translated separately by traversing their graphs. The additional conditions to match a pattern are inferred alongside. This pass utilizes the information from split tree inference and maps the symbolic split indices to runtime-obtained shapes of tensors. These three ASTs for Python functions are then composed to form an executable ONNX Script rewrite rule.

4 Implementation

This section introduces the major components of our implementation, challenges we met, and important technical choices. Table 1 lists and describes the operators our current implementation supports. Our artifact is available on GitHub [10].

4.1 Code Structure

Our code is modularized closely following the pipeline shown in Figure 1. The generation of candidate graphs is performed by the build function in synthesizer/build.py. To support operators with parameters, such as concat and conv, we follow the original implementation of TASO [9] that enumerates each parameter up to a prescribed bound. This can

Name	Description	Parameters
add	Element-wise addition	
matmul	Batched matrix multiplication. The dimensions before the last two are batching dimensions.	
concat	Concatenation along axis	axis
split	Split into two tensors along axis. Has two outputs.	axis
conv	Convolution with stride.	stride

Table 1. Operators supported by our implementation.

also be perceived as treating each operator with different parameters as a distinct operator.

onnx_gen/concat_info_inference.py performs split tree inference. For every graph pattern, create an instance of the class ConcatInfoInference and call the infer_all() member function on the graph. After that, the needed information, including split dimensions and symbolic split positions for the split operator, is stored in the same class instance.

The generated patterns are then categorized roughly according to their hash values and patterns within the same category are validated with their results on random inputs. synthesizer/fingerprint.py evaluates each graph on some random but globally fixed integer tensors. The hashes of outputs are used as the hashes of graphs. synthesizer/validate.py then tests pairs of graphs within the same category with some random floating-point tensors. Graphs with the same results (up to some small ϵ for floating-point numbers) are considered semantically valid.

To get the profitability of a rewrite, we run its target and replacement patterns respectively on random input tensors and measure the execution time. Currently, this simple evaluation is done with NumPy and CPU. However, we claim that our program is extensible to adopt other execution environments, cost models, and rule selection strategies.

With the information from split-tree inference, onnx_gen/rule_gen.py translates the internal representation of rewrite rules to executable rewrite rules in ONNX Script. It translates the target and replacement graphs respectively to Python ASTs for ONNX functions. The symbolic split positions are mapped to runtime values of tensor shapes. Furthermore, for patterns with multiple output operators, match_condition_gen.py generates conditions that require the matched outputs to be distinct operators. This prevents matching different outputs of a pattern against the same operator in a graph, which leads to non-terminating application loops.

4.2 Technical Decisions

We implemented the entire system in Python. This enables us to leverage the existing tensor coomputation libraries, including NumPy, and symbolic libraries, i.e., SymPy, to do

rewrite validation and split-tree inference. This also facilitates the generation of ONNX Script rewrite rules as Python ASTs structurally.

Initially, we planned to integrate our generated rules into XLA [8]. However, we came into two major problems:

1. XLA’s structural representation is only up to graph patterns. It does not support a uniform representation for rewrite rules. All matching and substitution logic must be written in C++ ad hoc.
2. XLA’s Match can match only single-root graph patterns. To implement multi-root patterns, one must find and iterate through the siblings of the current root operator manually.

These two drawbacks collectively make the automatic translation to executable rewrites highly challenging with too many corner cases. This is aggravated by the long build time of XLA, which renders the debugging process more difficult.

Based on these considerations, we chose ONNX [6] and ONNX Script [5] as our backend. ONNX is a format for the interoperability and translation between NNs written in different deep learning frameworks, including PyTorch and TensorFlow. ONNX Script enables building ONNX models in idiomatic Python and their manipulation. Specifically, ONNX Script allows building a graph rewrite rule with 3 functions, respectively for the target pattern, replacement pattern, and matching condition. These functions can then be built into a rewrite rule by applying `pattern.RewriteRule`.

All these components can be constructed as Python ASTs. The compositionality of ASTs allows a clear modularization of the construction of the 3 parts in our implementation. Furthermore, Python ASTs are more structural as a target of code generation comparing with strings of C++ source code.

5 Experiment

The instructions on how to run our experiment are available in the `README.md` of our repository. Our experiment environment is a Macbook Pro with a 10-core M2 Pro CPU and 16 GB main memory.

5.1 Results

We identified important rewrite rules from the generated rules. They are picked by their fingerprints and examined manually. These rules include the matrix multiplication fusion shown in the original TASO paper [3] (Figure 2a) and the distributivity of convolution over addition, which eliminates one high-cost convolution operation (Figure 2b).

We evaluate the benefits of our automatically generated rules with a 2-layer MLP classifying the MNIST dataset. The model is implemented in PyTorch. To apply the rewrite rules, we convert it to an ONNX model, apply the rewrites, and convert it back to PyTorch model. We run both the original and optimized models on the MNIST test set for 5 times and

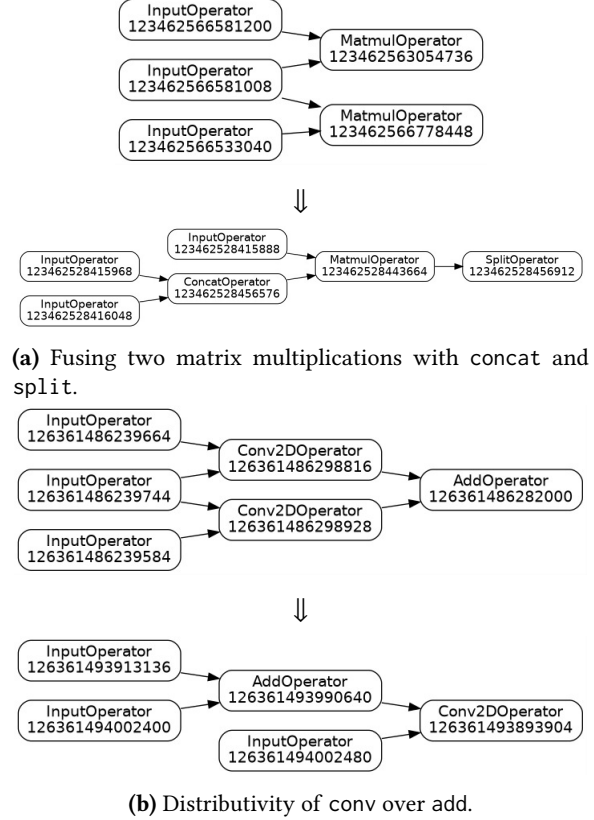


Figure 2. Rewrite rule examples. Generated by graphviz, which is not order-sensitive and may reorder nodes.

Model type	Inference time / s	Improvement
Original MLP	1.700	0.35%
Optimized MLP	1.694	

Table 2. Inference time and acceleration of MLP on the MNIST test set.

take the average running time respectively. The results are shown in Table 2.

We also tried to experiment on LeNet5. Our implementation fully supports the search, validation and code generation to ONNX Script for conv. However, PyTorch and ONNX are incompatible concerning the `kernel_shape` of conv operators. As a result, the ONNX model for our LeNet5 cannot be loaded back to PyTorch even without any transformations, thus we cannot test models with conv.

5.2 Interpretation of Results

The generated rewrite rules achieves a 0.35% average speedup on 5 inference runs. This is observable but not significant. This may arise from two causes:

1. Small test model size. We measured the performance only on a 2-layer MLP due to limited time. The benefits of graph rewrites may not be significant enough because of the limited number of locations that can be optimized.
2. Modern machine learning frameworks, like PyTorch, are already highly optimized internally. Therefore, as a method dating back to 2019, TASO may not give a portion speedups comparable with that time.

If given the opportunity to further develop this project, we will try to fix the toolchain to enable more comprehensive experiments. Furthermore, we will implement more sophisticated cost models and rule searching strategies to fully utilize the large set of generated rewrite rules.

References

- [1] Roy Frostig, Matthew Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. In <https://mlsys.org/Conferences/doc/2018/146.pdf>.
- [2] Zhihao Jia. 2019. Metaflow_sysml19 repository for sysml19 artifacts evaluation. https://github.com/jiazhihao/metaflow_sysml19. Accessed: 2025-12-18. (2019).
- [3] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19)*. Association for Computing Machinery, Huntsville, Ontario, Canada, 47–62. ISBN: 9781450368735. doi:10.1145/3341301.3359630.
- [4] Zhihao Jia, James Thomas, Todd Warszawski, Mingyu Gao, Matei Zaharia, and Alex Aiken. 2019. Optimizing dnn computation with relaxed graph substitutions. In *Proceedings of Machine Learning and Systems*. A. Talwalkar, V. Smith, and M. Zaharia, (Eds.) Vol. 1, 27–39. https://proceedings.mlsys.org/paper_files/paper/2019/file/4dd1a7279a8cfeea2660fbc34f02a2bc-Paper.pdf.
- [5] 2023. Onnx script: authoring onnx with the ease of python. <https://microsoft.github.io/onnxscript/>. Accessed: 2025-12-15. (2023).
- [6] 2017. Onnx specification. <https://onnx.ai/>. Accessed: 2025-12-15. (2017).
- [7] 2019. *Pytorch: an imperative style, high-performance deep learning library*. *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, 12 pages.
- [8] Amit Sabne. 2020. Xla : compiling machine learning for peak performance. (2020).
- [9] 2019. Taso: the tensor algebra superoptimizer for deep learning. <https://github.com/jiazhihao/TASO>. Accessed: 2025-12-18. (2019).
- [10] Heng Zhong, Bhavya Hirani, and Mengyao Wang. 2025. Cs521_taso. https://github.com/heng-zhong-2003/CS521_TASO. Accessed: 2025-12-16. (2025).

A Division of Work

Heng Zhong. Proposal; progress meeting slides and presentation; final report; operators and graph class architecture; add, matmul, concat, split operators; graph evaluation, fingerprinting, and validation; split tree inference; ONNX Script code generation; whole system integration. About 1800 lines of code out of 3000 + all writing tasks.

Bhavya Hirani. Rule generation; operators and graph classes debugging; graph evaluation, fingerprinting, and validation; conv operator and the inference passes thereon; experiment. About 700 lines of code.

Mengyao Wang. Provisional codegen to XLA; cost modeling; concrete shape inference; experiment scaffolding. About 500 lines of code.

B Complete Rules for Split Tree inference

Notations. We denote tensors as capital letters $A, B, \dots$. The shape of a tensor A is represented with a list $[d_A^{(0)}, \dots, d_A^{(n-1)}]$, where n is the rank of A and $d_A^{(i)}$ is the size of its i -th dimension. The *split info* of A is a list of pairs $\langle i, s \rangle$. This means that, the next time when A is splitted, it has to be splitted along its i -th dimension at the s -th element of the last $\langle i, s \rangle$ pair of its split-info list. Defining split info as a list enables nested concatenation.

As auxiliary functions, we let $\text{shape}(A)$ be the shape of A , $\text{rank}(A)$ be the rank of A , and $\text{info}(A)$ the split info associated with A . We also use the notation $l :: x$ for $[l_0, \dots, l_{n-1}, x]$, i.e., appending element x to list l .

Rules. We give the rules for split-tree inference in Figure 3. Note that all shapes are symbolic values (implemented with SymPy) and all ranks are actual integers. We omit the inference rules for symbolic and assume them as understood.

Note that we disallow inputs to the conv operator to be the result (either direct or indirect) of some concat operator. This is because conv contracts by summation at the concatenation border; we cannot recover the effect of conv with a split.

$$\begin{array}{c}
\frac{\text{info}(A) = l :: x \quad \text{info}(B) = \text{nil} \quad \text{shape}(A) = \text{shape}(B)}{\text{info}(\text{add}(A, B)) = l :: x} \text{ ADD-L} \\
\\
\frac{\text{info}(A) = \text{nil} \quad \text{info}(B) = l :: x \quad \text{shape}(A) = \text{shape}(B)}{\text{info}(\text{add}(A, B)) = l :: x} \text{ ADD-R} \\
\\
\frac{\text{rank}(A) = \text{rank}(B) \quad \text{info}(A) = \text{nil} \quad \text{info}(B) = l :: \langle \text{rank}(B) - 1, p \rangle \quad \text{shape}(A)[-2] = \text{shape}(B)[-2] \quad \text{shape}(A)[-2] = \text{shape}(B)[-1]}{\text{info}(\text{matmul}(A, B)) = l :: \langle \text{rank}(B) - 1, p \rangle} \text{ MATMUL-R} \\
\\
\frac{\text{rank}(A) = \text{rank}(B) \quad \text{info}(A) = l :: \langle \text{rank}(A) - 2, p \rangle \quad \text{info}(B) = \text{nil} \quad \text{shape}(A)[-2] = \text{shape}(B)[-2] \quad \text{shape}(A)[-2] = \text{shape}(B)[-1]}{\text{info}(\text{matmul}(A, B)) = l :: \langle \text{rank}(A) - 2, p \rangle} \text{ MATMUL-L} \\
\\
\frac{\text{rank}(A) = \text{rank}(B) \quad \text{shape}(A)[i] = \text{shape}(B)[i] \quad \text{info}(A) = l \quad \text{info}(B) = \text{nil}}{\text{info}(\text{concat}(A, B, \text{axis} = i)) = l :: \langle i, \text{shape}(A)[i] \rangle} \text{ CONCAT-L} \\
\\
\frac{\text{rank}(A) = \text{rank}(B) \quad \text{shape}(A)[i] = \text{shape}(B)[i] \quad \text{info}(A) = \text{nil} \quad \text{info}(B) = l}{\text{info}(\text{concat}(A, B, \text{axis} = i)) = l :: \langle i, \text{shape}(B)[i] \rangle} \text{ CONCAT-R} \\
\\
\frac{\text{info}(A) = l :: \langle i, p \rangle}{\text{info}(\text{split}_0(A)) = l} \text{ SPLIT-0} \quad \frac{\text{info}(A) = l :: \langle i, p \rangle}{\text{info}(\text{split}_1(A)) = l} \text{ SPLIT-1}
\end{array}$$

Figure 3. Rules for split-tree inference